Unit 2

Functions in C++

2.1 Functions in C++

What is a Function?

A function is a named, self-contained block of code that performs a specific task. Functions allow for:

- Code modularity
- Reusability of logic
- Easier debugging and testing
- Better program structure

Types of Functions:

Library Functions	Built-in, like printf(), sqrt(), etc.
User-defined Functions	Created by the programmer for specific tasks

- Syntax of User-defined Function:

```
return_type function_name(parameter_list) {
    // function body
}
```

Example: Function without parameters

```
#include <iostream.h>
void greet() {
   std::cout << "Welcome to C++ Functions!";
}

int main() {

   greet();  // function call
   return 0;
}
```

2.2 The main() Function

- Acts as the starting point of a C++ program.
- Can be void main() in Turbo C++ or int main() in standard C++.
- Only one main() function is allowed.

Example:

```
#include <iostream.h>
#include <conio.h>
int main() {
    std:: cout << "Inside main function.";
    return 0;
}
```

2.3 Function Prototyping

What is a Function Prototype?

A declaration of a function that tells the compiler:

- Function name
- Return type
- Number and type of arguments

- Syntax:

return_type function_name(parameter_list);

Placed before main() to ensure the compiler knows the function signature.

Example:

```
#include <iostream>

// Function prototype
void message();

int main() {

    message();
    return 0;
}

void message() {
    std::cout << "Prototype function works!";
}
```

Call by Reference

Passes the memory address of actual parameters to the function. Any change in formal parameters reflects in the actual ones.

Example:

```
#include <iostream>
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
Int main() {

    int x = 5, y = 10;
    swap(x, y);
    cout << "x = " << x << ", y = " << y;
    return 0;
}
```

Output:
Before swap: a = 10, b = 20
After swap: a = 20, b = 10

Changes in swap() affect a and b in main().

Return by Reference

Function returns a reference, not a copy. Useful when modifying and accessing the original data directly.

Example:

```
#include <iostream.h>
int value = 50;

int& getRef() {
   return value;
}
int main() {

   getRef() = 100;   // modifies original value
   std::cout << "Value: " << value;
   return 0;
}
```

Output:
Original value =50
Modified value = 100
getRef() returns a reference to value, allowing direct modification.

Concept	Purpose	Affects Original?	Use Case Example
Call by Reference	Function modifies original variable	✅ Yes	Swapping, updating
Return by Reference	Function returns actual variable	✅ Yes	Modify global/local var

2.5 Inline Functions

Why Inline?

- Saves time by avoiding function call overhead
- Best used for small functions

Syntax:

```
inline return_type function_name(parameters) {
   // function body
}
```

Example:

```
#include <iostream>
inline int cube(int n) {
   return n * n * n;
}
int main() {

   std::cout << "Cube = " << cube(3);
   return 0;
}
```

Normal Function

- When you call a normal function:
- The control jumps to the function's code.
- Then comes back to the calling place.

This jumping takes time and slightly slows down the program.

Inline Function

- When you call an inline function:
- The code of the function is copied at the place where it is called.

There is no jump, so it works faster for small functions.

2.6 Default & const Arguments

Default Arguments

If a value is not provided for a parameter, the default is used.

Example:

```
#include <iostream>


void add(int a, int b = 5) {
    std::cout << "Sum = " << a + b;
}

int main() {

    add(10);   // b = 5 used as default
    return 0;
}
```

const Arguments

- Used to protect parameters from modification
- Commonly used with references or pointers
- Example:

```
#include <iostream>



void display(const int x) {
    std::cout << "Value = " << x;
    // x = 20; // ❌ error: cannot change const
}

void main() {

    display(100);
    return 0;
}
```

Function	A reusable block of code
main()	Program entry point
Function Prototype	Tells compiler about function signature
Call by Reference	Modifies actual variables
Return by Reference	Returns actual memory reference
Inline Function	Replaces function call with actual code
Default Arguments	Provide fallback values
const Arguments	Prevent unintended modifications

2.7 **Function Overloading**

What is Function Overloading?

- Function Overloading means multiple functions can have the same name but with different parameters (type or number).

- It allows a programmer to perform different tasks using the same function name.

- **Syntax**:

```
return_type function_name(parameter list);
```

Same function name but different:

- Number of parameters
- Type of parameters
- Order of parameters

Example:

```
#include <iostream>


void print(int x) {
   std::cout << "Integer: " << x << "\n";
}

void print(float y) {
   std::cout << "Float: " << y << "\n";
}

void print(char ch) {
  std:: cout << "Character: " << ch << "\n";
}

void main() {

   print(100);       // Calls print(int)
   print(98.6f);     // Calls print(float)
   print('A');       // Calls print(char)
   return 0;
}
```

- All three functions are named print
- Each one accepts a different data type: int, float, char
- Compiler selects the correct function based on argument type

2.8. – **Friend Function:**

- A friend function is a non-member function that can access private or protected data of a class.
- Normally, only member functions can access a class's private data, but using friend, we allow an outsider function special permission.

- Imagine a locker (class) that is locked. Only the owner (member function) has the key.
- But sometimes, you trust a friend and give them the key — this is the friend function accessing private things.

Example:

```
#include <iostream>

class Student {
private:
   int marks;
public:
   Student() {
      marks = 90;
   }
   friend void show(Student s); // Declaration
};

// Friend function defined outside class
void show(Student s) {
   std::cout << "Marks (accessed by friend): " << s.marks << "\n";
}
void main() {

   Student s1;
   show(s1);  // Friend function can access private data
   return 0;
}
```

Element	Description
marks	Private data (normally inaccessible)
friend	Grants show() access to private members
show()	Defined outside the class, still can access marks

Use Useful when two classes/functions need tight access to each other's private data

2.9 – **Virtual Function**

- A virtual function is a member function in a base class that you expect to override in a derived class.

***You call a function using a base class pointer, but the derived class version of the function should execute.**

- A virtual function allows runtime polymorphism, meaning the correct function gets called based on the object, not the pointer type.

- **Why Use Virtual Functions?**
- To achieve Runtime Polymorphism (i.e., the correct function is selected at runtime, not compile time).
- Allows function overriding using pointers or references to base class.

- Imagine a remote (pointer) that can operate both a TV (base class) and a Set-top box (derived class).
- If the remote is pointing to the Set-top box, pressing a button should operate that — not

Method Overriding :

- A derived class provides its own version of a function that is already defined in its base class — with the same name and parameters.

- **Method Overriding vs Method Overloading**:

Feature	Overriding	Overloading
Relation	Between base and derived class	Within same class
Signature	Must be **same**	Must be **different**
Keyword used	`virtual` in base class	No special keyword
Purpose	**Runtime polymorphism**	Same function name for different tasks

Example: Method Overriding with virtual

```
#include <iostream>
using namespace std;

// Base class
class Animal {
public:
   virtual void sound() {
      std:: cout << "Animal makes a sound" << endl;
   }
};

// Derived class
class Dog : public Animal {
public:
   void sound() override {
       std::cout << "Dog barks" << endl;
   }
};

int main() {
   Animal* a;   // base class pointer
   Dog d;

   a = &d;
   a->sound();  // Output: Dog barks (method overridden)
```

```
#include <iostream>

class Base {
public:
   virtual void display() {
     std:: cout << "Display of Base Class\n";
   }
};

class Derived : public Base {
public:
   void display() {
      std::cout << "Display of Derived Class\n";
   }
};
int main() {
   Base *ptr;
   Derived d;
   ptr = &d;        // Base pointer points to Derived object
   ptr->display();   // Calls Derived class display (because display() is virtual)
   return 0;
}
```

Output:
Display of Derived Class

Without virtual, the output would be:
Display of Base Class

How It Works:

Base *ptr	A base class pointer
Derived d	A derived class object
ptr = &d	Pointer points to derived object
virtual void display()	Enables runtime function selection
ptr->display();	Calls Derived version at runtime

```
#include <iostream>
using namespace std;
// Base class
class Shape {
public:
   virtual void draw() {  // virtual function
      std::cout << "Drawing a generic shape." << endl;
   }
};
// Derived class
class Circle : public Shape {
public:
   void draw() override {  // override base class function
      std::cout << "Drawing a circle." << endl;
   }
};
// Another Derived class
class Rectangle : public Shape {
public:
   void draw() override {
      std::cout << "Drawing a rectangle." << endl;
   }
};
```

```
int main() {
   Shape* s;        // Base class pointer

   Circle c;
   Rectangle r;

   s = &c;          // Pointing to Circle object
   s->draw();       // Calls Circle's draw() – due to
virtual function

   s = &r;          // Pointing to Rectangle object
   s->draw();       // Calls Rectangle's draw()

   return 0;
}
```

Output:
Drawing a circle.
Drawing a rectangle.